# Algorithmics

# (CT065-3-3-ALG)

# Group Assignment 2

**Lecturer Name : Ms. Maznah binti Mohd Zin**

**Hand out Date : 20 April 2026**

**Hand in Date : 10 May 2026**

**Intake Code : APU3F2601CS(DA) / APD3F2601CS(DA)**

**Group No : 11**

**Group Members**

| Student ID | Student Name |
|---|---|
| TP068983 | Chan Chun Ming |
| TP068580 | Eu Jun Hong |
| TP068495 | Lim Beng Rhui (*Group Leader*) |
| TP067678 | Ng Xuan Jack |

## Table of Contents

**Workload Matrix** ····· 3
**Introduction** ····· 4
**Polynomial-Time (P) Problems** ····· 5
1.1 Concept and Identification of P Problems ····· 5
1.2 Time Complexity Analysis of Selected P Operations ····· 6
1.3 Practical Implications of P Problems in SmartLib ····· 8
**Non-Deterministic Polynomial-Time (NP) Problems** ····· 9
2.1 Concept and Identification of NP Problems ····· 9
2.2 Complexity Analysis of Selected NP Operation ····· 11
2.3 Limitations of NP Approaches in Real-Time Systems ····· 12
**Comparison between P and NP Problems in SmartLib Library** ····· 13
3.1 Comparison of P and NP based on Complexity and Scalability ····· 13
3.2 Smart Library Operations That Must Remain in P ····· 15
3.3 Operations That Can Tolerate Higher Complexity ····· 16
**Decision Making and Optimisation** ····· 17
4.1 Algorithm Selection: Optimal vs Approximate Recommendation System ····· 17
4.2 Real-World Optimization Technique for Handling NP Complexity ····· 19
**Conclusion** ····· 20
**References** ····· 21
**Appendix A. Java Implementation of P-Class Borrowing Operations in SmartLib** ····· 23

## Workload Matrix

| Task | Contribution Percentage | | | |
|---|---|---|---|---|
| | **Chan Chun Ming (TP068983)** | **Eu Jun Hong (TP068580)** | **Lim Beng Rhui (TP068495)** | **Ng Xuan Jack (TP067678)** |
| Introduction | - | - | 100% | - |
| Polynomial Time (P) Problems | - | 100% | - | - |
| Non-Deterministic Polynomial Time (NP) Problems | - | - | 100% | - |
| Comparison between P and NP Problems | - | - | - | 100% |
| Decision Making and Optimisation | 100% | - | - | - |
| Conclusion | 100% | - | - | - |
| **Overall Contribution** | **27.5%** | **22.5%** | **27.5%** | **22.5%** |

## Introduction

As the world moves into an increasingly digitalised era, different types of systems, ranging from research-intensive platforms that are used for critical tasks like space exploration, to mobile applications like banking systems that have to handle millions of transaction records everyday, are facing challenges in providing real-time responses within seconds while handling large amounts of data. Considering the importance of low response time, not only to guarantee operational efficiency and data usability but also to provide enhanced user experience, the computational complexity of algorithms, which indicates how efficiently an algorithm is able to handle and process workloads, plays an important role in modern systems as it provides an estimation towards the time taken to complete each task within the system. From a theoretical aspect, problems that are associated with computational complexity are classified into two, namely polynomial-time (P) problems and non-deterministic polynomial-time (NP) problems, where each has their own characteristics that influence their practicality in real-world scenarios.

To evaluate the differences between the two categories of computational complexity, this report will investigate the P and NP problems that exist in SmartLib. As a university library system that handles a high volume of book records with various functionalities, including book searching, popular book tracking and personalised book recommendations for students, the SmartLib system contains several functions that can be categorised into P and NP problems and are suitable to be analysed based on their computational complexity, thereby uncovering the strengths and potential limitations that exist within the different functionalities of the SmartLib system. With these insights, suitable recommendations can also be outlined to allow the system to deal with computationally extensive problems more effectively, hence enhancing the response time and throughput of the system while handling the large number of concurrent operations from students and lecturers.

## Polynomial-Time (P) Problems

This section outlines the characteristics of polynomial-time (P) problems, together with an analysis towards the functionalities in SmartLib that are classified under this category.

### 1.1 Concept and Identification of P Problems

In computational complexity, polynomial-time (P) problems refer to a class of decision problems that can be solved by a deterministic Turing machine in polynomial time. In other words, for algorithms classified under P problems, their execution time can be represented as a polynomial function of the input size, such as $O(n)$ and $O(n \log n)$. This indicates that the number of operations involved in algorithms will grow at a manageable rate as the input size increases, allowing these algorithms to have a stable and efficient performance when handling a large amount of input data. Hence, algorithms classified as P problems are considered to be computationally tractable (Adhikari et al., 2025), making them highly suitable to be applied in real-time systems that require fast and predictable performance.

In the context of the SmartLib system, several operations that require fast, consistent and scalable performance are classified as P problems, where some of them include:

- **Retrieval of student borrowing history**

  The operation of retrieving the borrowing history of a specific student can be classified as a P problem, as the system uses unique student identifiers to perform a direct search query by implementing mechanisms like indexed lookup. This allows the query operation to efficiently extract and display the associated student borrowing records without involving any exponential or combinatorial evaluations. Especially in SmartLib that handles 15,000 students with each of them having an average borrowing history of 30 books, maintaining the operation as a P problem allows it to perform effectively at low computational costs even when handling hundreds of thousands of records.

- **Recording new borrowing transaction**

  Since the SmartLib system has to handle an average of 500 daily book borrowing operations and 450 daily book return operations, it is necessary for the system to instantly record new borrowing entries and update the book's availability to ensure that each record is processed accurately and efficiently. Since the task primarily involves inserting new records and updating the database via a fixed sequence of operations, the borrowing transaction recording operation is hence classified as a P problem due to its low cost of computation and predictable performance, even when the transaction size increases.

## 1.2 Time Complexity Analysis of Selected P Operations

Based on the selected operations, time complexity analysis is performed to understand their performance under different scenarios:

- **Retrieval of student borrowing history**

  Based on Figure 1, several sequential operations are involved when retrieving students' borrowing history from the SmartLib system. The algorithm begins with accessing the corresponding `Student` object from the database implemented using a `HashMap` that supports direct key-based indexing and yields a time complexity of $O(1)$. After ensuring that the `Student` object is retrieved correctly, the algorithm then proceeds with extracting the corresponding borrowing history of the student, which requires $O(1)$ time since the operation is a direct reference retrieval. Lastly, to display each borrowing record, the system iterates through the list of borrowing history, resulting in a linear time complexity of $O(k)$, where $k$ represents the number of borrowing records of the student. These operations lead to the overall time complexity of:

$$O(1) + O(1) + O(k) = O(k)$$

  Since the execution time increases linearly with the number of borrowing history of students, the retrieval of student borrowing history can be classified as a P problem.

**Figure 1**

*Code Snippet for Student Borrowing History Retrieval*

```
public void retrieveBorrowingHistory(String studentId) {

    // First, retrieve the student from the database (hashmap) - O(1)
    Student student = studentDatabase.get(studentId);
    if (student == null) {
        System.out.println("Student not found.");
        return;
    }

    // Print the details of the student
    System.out.println("\nBorrowing History for: " + student.getStudentName());
    System.out.println("Student ID: " + student.getStudentId());

    // Retrieve the borrowing history - O(1)
    List<BorrowRecord> history = student.getBorrowingHistory();
    if (history.isEmpty()) {
        System.out.println("No borrowing records found.");
        return;
    }

    // Retrieve and print each borrowing record - O(k) where k = number of borrowing history
    for (BorrowRecord record : history) {
        System.out.println(record);
    }
}
```

- **Recording new borrowing transaction**

  As shown in Figure 2, recording a new borrowing transaction also involves a series of sequential operations. Similar to the borrowing history retrieval operation, the algorithm first uses student ID to perform a direct lookup on the student database implemented using `HashMap`, which operates under a constant time complexity of $O(1)$. Once making sure that the student exists in the system, the algorithm then checks for book availability by performing another direct lookup using book ID that also requires $O(1)$ time. If the book is available for borrowing, the system will then proceed with creating a new borrowing record, updating the book quantity by decrementing it, and lastly appending the record to the student's corresponding borrowing list. As each operation only involves direct access and fixed-step updates, these operations are executed under a constant time complexity of $O(1)$ as well. Hence, the overall time complexity can be expressed as:

$$O(1) + O(1) + O(1) + O(1) + O(1) = O(1)$$

Since the algorithm's execution time is independent from its input size, such as the number of users and books in the system, this indicates that the recording of borrowing transaction is highly efficient and predictable. Therefore, the operation, suitable to be implemented for real-time processing, is also categorized as P problems.

**Figure 2**

*Code Snippet for Recording New Borrowing Transaction*

```
public void recordNewBorrowingTransaction(String studentId, String bookId) {

    // Retrieve the student from the database - O(1)
    Student student = studentDatabase.get(studentId);

    // Check if the student exists - O(1)
    if (student == null) {
        System.out.println("Student not found. Transaction failed.");
        return;
    }

    // Retrieve the number of stocks available for the book - O(1)
    BookStock stock = bookInventory.get(bookId);

    // Check if the book is available - O(1)
    if (stock == null || stock.getQuantity() <= 0) {
        System.out.println("Book is currently not available. Transaction failed.");
        return;
    }

    // Create borrowing record by retrieving book using book ID, then decrease its quantity - O(1)
    BorrowRecord newRecord = new BorrowRecord(this, bookId);
    stock.decreaseQuantity();

    // Add the borrowing record to the student's borrowing history and print successful message - O(1)
    student.addBorrowRecord(newRecord);
    System.out.println("\nNew borrowing transaction recorded successfully.");
}
```

### 1.3 Practical Implications of P Problems in SmartLib

The two operations analysed, namely the retrieval of student borrowing history and the recording of new borrowing transactions, have a polynomial time complexity that allows these operations to perform efficiently with high predictability and scalability. Therefore, P problems are suitable to be implemented in real-time systems like the SmartLib system, especially under scenarios such as:

- **2-second response requirement**

  As the SmartLib system imposes a strict real-time constraint where the search results must be returned within 2 seconds, which involves operations like borrowing history retrieval and book search queries, it is necessary for these features to remain in polynomial time complexity so that SmartLib can provide fast and consistent responses even when handling high amount of users during peak hours. By incorporating different mechanisms, such as `HashMap` to support direct lookups with $O(1)$ time complexity, regardless of their input size, these P problem-associated operations will not require excessive computational costs and resources, thereby minimising user interface latency while ensuring that these operations can satisfy the real-time performance requirements of the SmartLib system.

- **High number of daily operations**

  Since the SmartLib system also handles a massive volume of daily operations, such as an average of 2000 search queries, 500 borrowing and returning transactions every day, especially during peak hours where a high number of users concurrently access the system for different operations, SmartLib must be equipped with the capability to accurately and efficiently process these events. By having its core functionalities to remain as P problems, the algorithm of these features will be able to gain a manageable growth rate in terms of number of operations and execution time, thereby allowing the system to be able to effectively utilise its CPU and memory resources while reducing any potential computational overhead. Not only this prevents the SmartLib system from encountering performance bottlenecks and system slowdowns, but also ensures that the system is able to maintain a scalable, consistent, and highly responsive performance when SmartLib has to handle high volume of transactions during peak periods.

## Non-Deterministic Polynomial-Time (NP) Problems

This section outlines the definition and characteristics of NP problems, together with an analysis towards the functionality associated with NP problems in the SmartLib system.

### 2.1 Concept and Identification of NP Problems

According to Wang (2025), a problem is defined as non-deterministic polynomial-time (NP) if it satisfies two conditions, where:

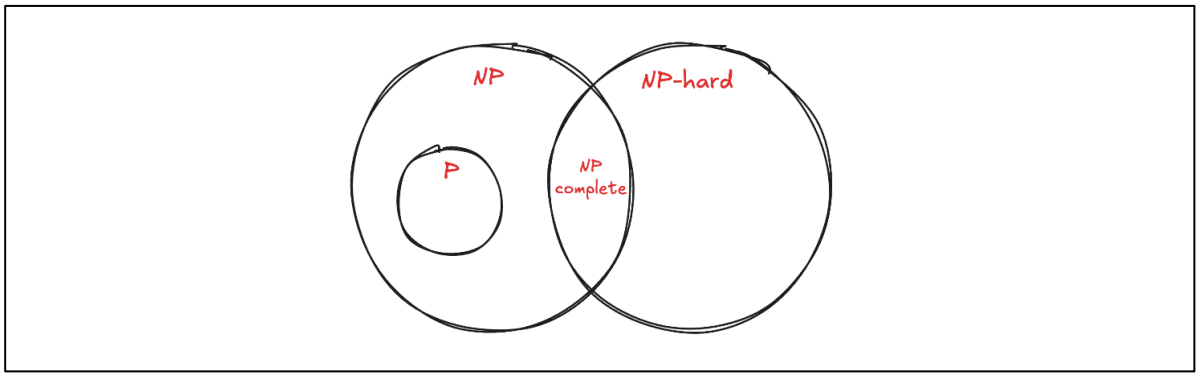
- **The problem can be solved in a non-deterministic polynomial time.**
  The process of solving an NP problem is often non-deterministic, where theoretically speaking, the non-deterministic machine will be able to explore different solution paths simultaneously to identify its solution in polynomial time. However, since programs are executed on deterministic machines, it is impossible for them to simultaneously explore different possibilities during practical implementations. Hence, determining the optimal solution for NP problems often requires the algorithm to iterate through a large number of possibilities and choices, making the entire process computationally expensive and resulting in exponential or combinatorial time complexity.

- **The solution to the problem can be verified in polynomial time.**
  If a solution to the NP problem is provided, the correctness of the solution can be validated within polynomial time using a deterministic machine. In other words, the system will only have to apply rule-based checks to confirm that the solution satisfies the problem requirements without having to regenerate all the possible outcomes, making the solution verification process to be efficient and scalable despite the computational expensiveness of NP problems when finding for the optimal solution.

In simple terms, a problem is considered as NP if its solution is verifiable in polynomial time but requires significantly higher computational effort to obtain the optimal solution. Extending from NP problems, another problem category, namely the NP-hard problems, is also derived, where according to Shiwakoti (2025), NP-hard problems are “at least as difficult” as the hardest problems in NP. The Venn diagram in Figure 3 depicts the relationship between NP and NP-hard problems, where although NP-hard problems also require the algorithm to evaluate a large number of possible combinations to determine the optimal solution, NP-hard does not ensure that their solutions are be verifiable in polynomial time.

**Figure 3**

*Relationship between Different Classes of Computational Complexity*




*Note*. From Understanding P, NP, NP-complete, and NP-hard problems, by C. Wang, 2025 (https://medium.com/@michaelclion/understanding-p-np-np-complete-and-np-hard-problems-f09a2b09cbf1).

From the perspective of the SmartLib system, its personalized book recommendation algorithm is considered as an NP-hard problem, as it resembles the famous NP-hard 0/1 knapsack problem (Cifci, 2023), where the algorithm will have to evaluate a large number of possibilities to find the best recommendation set under a fixed recommendation limit. Each element of the 0/1 knapsack problem can be mapped to the recommendation algorithm, as shown in Table 1.

**Table 1**

*Mapping between 0/1 Knapsack Problem and Recommendation Algorithm in SmartLib*

| Knapsack Element | Recommendation Equivalence | Explanation |
|---|---|---|
| Items | Books | Books are treated as items that may or may not be chosen for recommendation, similar to the items in the knapsack problem. |
| Value | Recommendation score | Each book has its own recommendation value that varies based on multiple factors, such as borrowing history and genre preferences, aligning with the value of each item in the knapsack problem. |
| Weight | Recommendation constraints | Recommendations are constrained by several factors, such as book availability and mismatching user preferences, similar to the weight restriction in the knapsack problem. |
| Capacity | Recommendation slots | Only a fixed number of books will be selected and recommended to users, which is similar to the knapsack problem with a capacity fixed. |
| Optimisation goal | Recommendation quality | The algorithm attempts to find the best recommendation quality based on multiple conditions while satisfying system requirements, similar to the knapsack problem where the total value is maximised. |

As the SmartLib system manages 100,000 books, the algorithm will have to perform evaluation on an extremely large number of possible book combinations and perform ranking operations based on their recommendation scores before determining the best recommendation set for each user. If the system continues to expand, where the number of books, user borrowing history and recommendation requests increase, the algorithm's computational cost will increase combinatorially, making the overall recommendation pipeline computationally expensive to be executed. Therefore, since the recommendation algorithm adopts an optimisation structure similar to the NP-hard 0/1 knapsack problem, the personalised book recommendation feature in the SmartLib system can be categorised as an NP-hard problem.

## 2.2 Complexity Analysis of Selected NP Operation

As the recommendation algorithm will have to evaluate different book combinations to identify the most appropriate suggestion for users, by letting *n* represent the number of books in the system and letting *k* represent the number of books to be recommended by the system, the algorithm will have to determine the best *k* books from *n* available books. In mathematical terms, this represents a combinatorial selection process that resembles a time complexity of:

$$O\left(\binom{n}{k}\right)$$

In the theoretical worst-case scenario, the algorithm will have to exhaustively evaluate all possible combinations within a combinatorial search space of $\binom{n}{k}$ and apply a scoring function to measure the relevance of each candidate subset, so that the most optimal recommendation can be identified and suggested by the algorithm. Since the time complexity of the algorithm grows rapidly as the search space increases, this makes the algorithm to be computationally expensive and inefficient, especially under scenarios where:

- **The number of books in the system increases**
  As the size of $n$ increases, the number of possible subsets increases combinatorially, thereby drastically increases the size of the search space and requires the algorithm to perform a significantly higher number of scoring operations.

- **The number of borrowing history of student grows**
  As user borrowing history increases, the algorithm will have to handle and utilise more contextual and behavioural data to capture user preferences, hence increasing the computational cost of the scoring function when evaluating each candidate subset.

### 2.3 Limitations of NP Approaches in Real-Time Systems

As the NP-hard personalised recommendation algorithm has a combinatorial time complexity that significantly increases the computational cost and time when the search space expands, especially in real-time systems that prioritises output timeliness and responsiveness (Intel, n.d.), this makes the recommendation algorithm not suitable and practical to be applied in the context of the SmartLib system, especially under scenarios like:

- **Handling 300 system recommendation requests per day**

  As the algorithm has to evaluate a large number of possible book combinations to determine the most optimal recommendation set for each request, the majority of system resources, such as CPU processing power and memory, will be occupied by the algorithm to process these calculations. Especially with 300 daily recommendation requests, such a high workload forces the SmartLib system to be executed under computational pressure, thereby introducing processing delays and system bottlenecks when users simultaneously access other functionalities like book search, availability checking or borrowing request.

- **Future scalability of the SmartLib system**

  As the SmartLib system will continue to scale up in terms of number of books, user borrowing history and recommendation requests, due to the combinatorial explosion nature of the NP-hard recommendation algorithm, this significantly expands the search space and causes the algorithm to spend additional time to evaluate a higher number of candidate subsets before being able to identify the most optimal recommendation set for users. If appropriate optimisation techniques are not applied to the recommendation algorithm, the SmartLib system will face difficulty in maintaining a low response time and stable performance when handling user requests, making the system inefficient and infeasible to be deployed in large scale.

**Comparison between P and NP Problems in SmartLib Library**

This section compares the characteristics between polynomial-time (P) and non-deterministic polynomial-time (NP) problems, so that the performance requirements for each functionality in the SmartLib system can be identified and outlined.

## 3.1 Comparison of P and NP based on Complexity and Scalability

The complexity and scalability difference between P and NP problems are evaluated based on the following aspects:

- **Time complexity**

  The main difference between P and NP problems lies in their time complexity, where P problems can be solved in polynomial time, while NP problems often need significantly higher computational costs and resources to obtain the most optimal solution. The reason behind is that P problems often have time complexities like $O(n)$ and $O(n \log n)$, in which the number of operations performed by the algorithm grow at a manageable rate as the input size grows. On the other hand, the search space of NP problems with exponential or combinatorial time complexities like $O(2^n)$ increases drastically as the input size increases, making these problems computational expensive (Boldori, 2024).

- **Execution time**

  Due to the polynomial time complexities of P problems, their execution time often remains predictable and efficient under different input sizes, thereby allowing real-time systems like SmartLib to fulfil system and performance constraints easily. However, NP problems often experience a longer and unstable execution time as finding the most optimal solution requires the algorithm to explore and evaluate a large number of possible solutions (Fortnow, 2009), thereby requiring approximation techniques or heuristic methods to be incorporated before implementing NP-related approaches in systems.

- **Scalability**

  As P problems are able to scale efficiently with increasing input sizes, this makes them suitable to be applied in systems that involve large volumes of data and frequent user interactions. In contrast, the exponential or combinatorial growth of computational requirements makes NP problems often scale poorly and inefficiently when input size changes (Li et al., 2020), which adversely impacts system throughput, performance and responsiveness as the system expands and handles more and more data.

- **Solution Verification**

  In terms of solution verification, both P and NP problems are similar, where although NP problems often require a much longer time to discover its optimal solution, verifying a given solution of both problem classes can be done within polynomial time. Since the solution for P problems can be obtained in polynomial time, it is often straightforward to verify the solution and determine its correctness. However, for NP problems, although getting the optimal solution is computationally difficult, its efficient solution verification process makes trial-and-error strategies feasible to be applied, supporting the efficient evaluation of candidate solutions to identify near-optimal results (Wan & Shi, 2010).

- **Practical Applicability**

  Practically speaking, P problems are generally more suitable for real-time systems as they provide a more predictable performance, efficient resource utilization, and consistent response time, even when handling large amount of input data, making them highly suitable for core system operations. On the other hand, the high complexity and unstable performance of NP problems make them less suitable to be applied for real-time systems, where approaches like heuristics and approximation techniques must be applied towards these problems to obtain a sufficiently accurate results within a specific timeframe (Aravkin et al., 2012) while balancing performance and computational complexity.

Overall, the difference between P and NP problems are summarised in Table 2.

**Table 2**

*Comparison Table between P and NP Problems*

| Aspect | P | NP |
|---|---|---|
| Time complexity | Solvable in polynomial time like $O(n)$, $O(\log n)$ and $O(n \log n)$ | Often involves exponential or combinatorial complexity like $O(2^n)$ |
| Execution time | Fast, efficient and predictable even with large input sizes | Slower and less predictable due to exploration of high number of possible solutions |
| Scalability | Scales efficiently for large systems and datasets | Scales poorly as computational requirements increase drastically |
| Solution Verification | Solutions can be solved and verified efficiently in polynomial time | Solutions can be verified in polynomial time, but finding them is computationally expensive |
| Practical Applicability | Suitable for real-time operations | Requires heuristics or approximation techniques in practical systems |

### 3.2 Smart Library Operations That Must Remain in P

To fulfil system and performance constraints while ensuring the satisfaction of users, some features of the SmartLib system must remain as P problems, which include:

- **Book search**

  To handle an average of 2000 search queries every day, the SmartLib system has to complete each book search within a time limit of 2 seconds. Due to the strict real-time requirements imposed, the book search feature must remain as P problems to ensure that when the number of books in the system increases, the execution time remains efficient and manageable. Therefore, when performing book searching using title, author or ISBN, efficient data structures like hash tables and indexed trees shall be implemented to allow the SmartLib system to perform fast lookup operations, thereby ensuring fast response time while minimising performance delays to provide a smooth user experience.

- **Borrowing and return updates**

  On average, the SmartLib system performs approximately 500 transactions for both book borrowing and returning operations. The high volume of transaction must be supported by efficient and consistent record updates, especially during peak hours to ensure system consistency and data accuracy while minimising bottlenecks. Hence, it is necessary for the borrowing and return update operations to remain as P problems so that the computational cost required is minimised to ensure immediate record processing.

- **Availability check**

  As part of the borrowing transaction, the book availability checking feature shall also remain as a P problem to allow users receive immediate response when performing the operation. By using techniques like hash table lookups, the algorithm is able to be executed in constant time, allowing it to have a fast and efficient performance without consuming excessive resources even when the number of books and users increases.

- **Popular book tracking**

  The algorithm to track the top 20 most popular books in real-time must also remain as a P problem to ensure that the system is able to efficiently update the counts when each borrowing transaction occurs. Hence, mechanisms like hash maps and priority queues shall be incorporated to update borrowing counts and maintain rankings effectively without having to recompute everything from scratch after new borrowing transaction.

### 3.3 Operations That Can Tolerate Higher Complexity

There are also several operations in the SmartLib system that do not require the fulfilment of strict real-time requirements due to the high computational complexity involved, which include:

- **Personalised recommendation generation**

  Due to the extremely large number of possible combinations among candidate items, this makes the personalised recommendation algorithm to be computationally intensive as the algorithm has to score and rank each book combination according to factors like user preferences and borrowing history. As more books are recorded into the SmartLib system, the search space will grow significantly, making the recommendation algorithm require additional computational resources to perform its operations while yielding inconsistent response time among users. These factors have made the personalised recommendation functionality to be less suitable for real-time systems like SmartLib. However, since users typically focus on item relevance rather than obtaining the most optimal recommendation, techniques like heuristics and approximation algorithm can be applied to reduce the search space of the algorithm while generating sufficiently relevant recommendations, making the recommendation algorithm more practical to be implemented in SmartLib.

- **Historical analysis of borrowing records**

  As an additional consideration, students' book borrowing records can be further utilised to perform historical analysis and understand different user behavioural patterns within the SmartLib system, such as borrowing behaviour, reading trends and user preferences, so that system recommendations can be enhanced while informative insights can be gained to support strategic decision making. Since historical analysis involves processing large volumes of data and does not require immediate response from users, although it introduces higher computational complexity, these processes will not interfere with real-time system functionalities as they can be executed through offline or batch processing techniques (Bobadilla et al., 2013). Hence, higher tolerance is acceptable for the feature as system robustness and responsiveness are not affected during day-to-day operations.

## Decision Making and Optimisation

This section recommends and justifies the choices of recommendation mechanism in the SmartLib system by outlining and discussing the differences between the fully optimal (NP) approach and approximation techniques, then followed by identifying and suggesting the appropriate optimization techniques to handle non-deterministic polynomial time (NP) complexity involved in the book recommendation algorithm.

### 4.1 Algorithm Selection: Optimal vs Approximate Recommendation System

From the perspective of a system designer, the approximate/heuristic recommendation mechanism is more suitable to be applied in the SmartLib system instead of the fully optimal (NP) approach. Although the NP approach is able to provide recommendations that are more accurate, implementing it into SmartLib is infeasible due to certain factors:

- **Computational Inefficiency**

  Due to the massive number of books available in the library, a fully optimal recommendation system will require extensive computational resources to evaluate the number of possible books combinations, and rank all possible recommendations based on borrowing history, user preferences, and relevance scores. This requires extra computational resources as the system needs to provide 300 recommendation requests per day, while supporting peak demand period with more than 500 concurrent users. The heavy computational overhead will reduce the throughput of the overall system, which significantly degrades the performance of other critical operations in SmartLib system, such as book searching, popular books tracking, and book availability checking.

  In contrast, heuristic-based algorithm is able to enhance system performance by narrowing down the combinations from all books into items that are more relevant to users, which significantly reduces the computation processes, database loads, and server resource consumption. In fact, the Bounded Rationality theory introduced by Simon (1957, as cited in Hernandez & Perez-Ortega, 2019) has indicated that humans could not evaluate every possible option to find the most optimal one due to cognitive limits such as time, environment, and attention. In the context of the SmartLib system, if the heuristic approach is able to provide a 90% relevant suggestion, when compared with the fully optimal approach that provides a 99% relevant recommendation, students are unlikely to discover any additional gain, hence justifying that extra optimization is unnecessary.

- **Strict Response Time Requirements**

  SmartLib system is designed as a real-time digital library system that handles a vast amount of students simultaneously. Hence, a strict temporal requirement is imposed on the system, where the search query results must be returned within two seconds. Since a fully optimal (NP) approach consumes high computational resources, latencies will be introduced when the system concurrently executes other functionalities like book search and availability checking. This reduces the overall responsiveness of the system and causes noticeable delays, resulting in poor user experiences when students expect instant results to be obtained, as the Doherty threshold suggests that a system should provide response within 400 milliseconds to enhance interactions and minimize frustrations (Kithmini, 2025). As users generally prefer fast and relevant recommendations rather than mathematically optimal ones, by applying approximation and heuristic mechanisms, such as simplified comparisons, filtering methods, and ranking heuristics, this allows personalized recommendations to be generated more instantly, thereby enhancing system throughput, providing smoother user interaction, and eliminating latency during peak periods. Hence, implementing approximation solutions is a more suitable choice to meet SmartLib's temporal requirements of 2 seconds while enhancing user satisfaction.

- **Future Scalability**

  Aside from enhancing system performance and response time, the approximate/heuristic approach also ensures the system's future scalability. As the SmartLib system expands, the recommendation system should also operate efficiently without major degradation in terms of response time, throughput, and user experience. If the fully optimal approach is implemented, the CPU power consumption and processing latency will increase combinatorically with the number of books available in the system, as shown by its time complexity of $O\left(\binom{n}{k}\right)$. Instead of searching and pairing all books, the heuristic approach can utilize techniques like genre filtering and popularity ranking to approximate relevant recommendations to users, hence avoiding high computational workloads while minimising the growth rate of resource consumption even when the database expands.

Overall, the approximate/heuristic approach is preferred to achieve the best balance between recommendation quality and operational performance, allowing the recommendation algorithm to be practical and feasible to be implemented in the SmartLib system.

### 4.2 Real-World Optimization Technique for Handling NP Complexity

One of the most effective optimisation techniques for handling NP complexity in the SmartLib system is via *precomputation with caching*. The feasibility of such an approach is proven by recent studies which outlined that caching enhances the response time of a system significantly (MacAvaney & Macdonald, 2025). In the SmartLib system context, this is achieved by generating recommendation results in advance during off-peak hours, where the system can periodically analyse student preferences and borrowing histories during low-demand periods, then store the results in the system's cache memory to allow instant retrieval when users request for recommendations. As a result, the system performance can be improved significantly in terms of:

- **Reduced real-time computational workload**
  By performing precomputation with caching, the system will have the recommendations available before users request for them, allowing it to only need to retrieve the results instead of repeating its computation for each request. This not only significantly reduces the CPU and memory usage during peak usage periods when the system has to handle more than 500 concurrent users, but also shifts heavy computational workloads to off-peak periods and balance resource utilization to reduce resource overhead. By so, the SmartLib system will be able to support simultaneous recommendation requests without major performance degradation.

- **Enhanced throughput and faster response time**
  With cached recommendations, the system's response time can be reduced significantly if compared to executing similar computations on every request. This is supported by the study from Latha and Rajaram (2010) who computed their system's response time when caching is implemented on their website, and it is found that the system's response time is significantly enhanced. The real-time responsiveness provided by precomputation with caching makes the recommendation algorithm suitable to be implemented in systems like SmartLib due to its high throughput and fast response time.

In short, precomputation with caching is particularly suitable for the SmartLib system. Since user borrowing behaviour does not substantially change every moment, the recommendation results remain useful for a reasonable period before requiring recalculation. Hence, by having recommendations generated during off-peak hours and store them in cache memory, the NP-hard problem is turned into a practical solution, supporting its implementation in SmartLib.

## Conclusion

In conclusion, the case study of the SmartLib system demonstrates the significance of computational complexity when designing real-world applicated systems. By identifying the characteristics and distinctions between polynomial-time (P) problems and non-deterministic polynomial-time (NP) problems, the report underscores the crucial role of P and NP problems in affecting system efficiency, responsiveness, and future scalability in real-world applications.

The analysis of P problems has shown that operations that grows proportionally with the input size such as borrowing history retrieval, book search, and availability checking are highly suitable for real-time implementation due to their predictable and deterministic outcomes. These functionalities should be maintained as polynomial-time processes to meet the system's strict 2-second response time deadline and ensure a consistent service delivery for students. In contrast, the personalised recommendation algorithm is identified as an NP-hard problem due to its combinatorial nature and resemblance to optimisation problems like the 0/1 knapsack problem. Although a mathematically optimal recommendation algorithm can provide the best suggestions to users, the computational resources required to evaluate combinations of 100,000 books is not realistic for real-time execution. This indicates the major challenge in computation complexity analysis when balancing the trade-offs between optimal solution and practical system constraints.

Furthermore, the comparison between P and NP problems also emphasised that core operational features of the SmartLib system such as book availability checks or book search must remain efficient and predictable using polynomial-time approaches, while high computational complexity can be mitigated in the personalised recommendation algorithm if it is supported by practical optimisation techniques. Based on these findings, the report has suggested the approximate/heuristic mechanism for the implementation of recommendation system in the SmartLib system. With optimisation techniques such as precomputation with caching, the system can deliver relevant recommendations instantly while maintaining the overall throughput of other system functionalities.

All in all, this study highlights the importance of having a sufficient understanding towards algorithmic complexity in system design. By implementing suitable algorithms based on theoretical complexity and practical performance requirements, the SmartLib system can enhance its performance by balancing between computational efficiency and user satisfaction, allowing it to become a reliable digital library system that can be adopted in universities.

## References

Adhikari, I. M., Gupta, S. P., & Dhungana, R. C. (2025). Computational problem and its time complexity. *The Nepali Mathematical Sciences Report, 42*(2), 72-90.

Aravkin, A., Friedlander, M., Herrmann, F., & Leeuwen, T. (2012). Robust inversion, dimensionality reduction, and randomized sampling. *Mathematical Programming: Series A and B, 134*(1), 101-125.

Bobadilla, J., Ortega, F., Hernando, A., & Gutiérrez, A. (2013). Recommender systems survey. *Knowledge-Based Systems, 46*, 109-132.

Boldori, A. (2024). p ≠ np: The set of deterministic problems that are solvable in polynomial time is unequal to the set of non-deterministic problems that are solvable in polynomial time. *Journal of Computer Science, 20*(10), 1263-1269.

Cifci, M. (2023, February 5). *NP-hard problems*. Retrieved from Medium: https://themanoftalent.medium.com/np-hard-problems-21852451488a

Fortnow, L. (2009). The status of the P versus NP problem. *Communications of the ACM, 52*(9), 78-86.

Hernandez, J. V., & Perez-Ortega, R. (2019). Bounded rationality in decision–making. *MOJ Current Research & Reviews*, 1-8. doi:10.15406/mojcrr.2019.02.00047

Intel. (n.d.). *What Is a real-time system?* Retrieved from Intel: https://www.intel.com/content/www/us/en/learn/what-is-a-real-time-system.html

Kithmini, S. (2025, January 25). *Understanding the Doherty threshold: A key UX law for superior user experience*. Retrieved from Medium: https://medium.com/@shakindikithmini/understanding-the-doherty-threshold-a-key-ux-law-for-superior-user-experience-d7fe0957d546

Latha, S. S., & M., R. (2010). Web caching and response time optimization based on eviction method. *International Journal of Engineering Science and Technology, 2*(9), 4912-4921.

Li, W., Ding, Y., Yang, Y., & Sherratt, R. (2020). Parameterized algorithms of fundamental NP-hard problems: A survey. *Human-centric Computing and Information Sciences, 10*(1), 1-24.

MacAvaney, S., & Macdonald, C. (2025). On precomputation and caching in information retrieval experiments with pipeline architectures. *Second International Workshop on Open Web Search*, 1-13. doi:https://doi.org/10.48550/arXiv.2504.09984

Shiwakoti, B. (2025, April 12). *P, NP, NP-complete, and NP-hard - Understanding computational complexity classes*. Retrieved from PSC Computer: https://balaramshiwakoti.com.np/blog/toc/p-np-complexity

Wan, C., & Shi, Z. (2010). A proof for P =? NP problem. *arXiv: Computational Complexity*.

Wang, C. (2025, March 7). *Understanding P, NP, NP-complete, and NP-hard problems*. Retrieved from Medium: https://medium.com/@michaelclion/understanding-p-np-np-complete-and-np-hard-problems-f09a2b09cbf1

# Appendix A

## Java Implementation of P-Class Borrowing Operations in SmartLib

```
import java.util.*;

/**
 * Represents a book in the library system.
 */
class Book {

    // Books have two attributes: Book
    private final String bookId;
    private String bookTitle;

    // Constructor to create a book
    public Book(String bookId, String bookTitle) {
        this.bookId = bookId;
        this.bookTitle = bookTitle;
    }

    // Getters and setters
    public String getBookId() { return bookId; }
    public String getBookTitle() { return bookTitle; }}


/**
 * Represents the stock of a book in the library system.
 */
class BookStock {

    // Record the book and its quantity
    private final Book book;
    private int quantity;

    // Constructor
    public BookStock(Book book, int quantity) {
        this.book = book;
        this.quantity = quantity;
    }

    // Getters and setters
    public Book getBook() { return book; }
    public int getQuantity() { return quantity; }

    // Functions to increase and decrease the number of books in stock
    public void increaseQuantity() { quantity++; }
    public void decreaseQuantity() {
        if (quantity > 0) quantity--;
    }
}


/**
 * Represents a borrowing record for a student.
 */
class BorrowRecord {

    // Attributes of a borrowing record
    private final Book book;
    private final Date borrowDate;
    private Date returnDate;
    private boolean returned;

    // Constructor
    public BorrowRecord(SmartLibSystem system, String bookId) {
        this.book = system.retrieveBook(bookId);
        this.borrowDate = new Date();
        this.returned = false;
    }

    // Mark the book as returned
    public void markReturned() {
        this.returnDate = new Date();
        this.returned = true;
    }

    // Print the borrowing record
    @Override
    public String toString() {
        return "Book ID: " + book.getBookId() +
                ", Title: " + book.getBookTitle() +
                ", Borrow Date: " + borrowDate +
                ", Returned: " + returned +
                (returned ? ", Return Date: " + returnDate : "");
    }
}
```

```java
/**
 * Represents a student in the library system.
 */
class Student {

    // Attributes of a student
    private final String studentId;
    private String studentName;
    private List<BorrowRecord> borrowingHistory;

    // Constructor
    public Student(String studentId, String studentName) {
        this.studentId = studentId;
        this.studentName = studentName;
        this.borrowingHistory = new ArrayList<>();
    }

    // Getters and setters
    public String getStudentId() { return studentId; }
    public String getStudentName() { return studentName; }
    public void setStudentName(String studentName) { this.studentName = studentName; }
    public List<BorrowRecord> getBorrowingHistory() { return borrowingHistory; }
    public void setBorrowingHistory(List<BorrowRecord> borrowingHistory) { this.borrowingHistory = borrowingHistory; }

    // Add borrowing record to student's borrowing history
    public void addBorrowRecord(BorrowRecord record) { borrowingHistory.add(record); }
}

/**
 * Smart Library System for managing student borrowing operations.
 */
class SmartLibSystem {

    // Inventory of books and database of students
    private final Map<String, BookStock> bookInventory;
    private final Map<String, Student> studentDatabase;

    // Constructor
    public SmartLibSystem() {
        bookInventory = new HashMap<>();
        studentDatabase = new HashMap<>();
    }

    // Add student to the system
    public void addStudent(Student student) {
        studentDatabase.put(student.getStudentId(), student);
    }

    // Add a book to the inventory
    public void addBookToInventory(Book book) {

        // If book is already in inventory, just increment its quantity
        if (bookInventory.containsKey(book.getBookId())) {
            bookInventory.get(book.getBookId()).increaseQuantity();

        // If not, add the book to the inventory and set its quantity as 1
        } else {
            bookInventory.put(book.getBookId(),new BookStock(book, 1));
        }
    }

    // Operation 1: Retrieve the borrowing history for a student
    public void retrieveBorrowingHistory(String studentId) {

        // First, retrieve the student from the database (hashmap) - O(1)
        Student student = studentDatabase.get(studentId);

        // Check if the student exists - O(1)
        if (student == null) {
            System.out.println("Student not found.");
            return;
        }

        // Print the details of the student
        System.out.println("\nBorrowing History for: " + student.getStudentName());
        System.out.println("Student ID: " + student.getStudentId());

        // Retrieve the borrowing history - O(1)
        List<BorrowRecord> history = student.getBorrowingHistory();

        // Check if the history is empty
        if (history.isEmpty()) {
            System.out.println("No borrowing records found.");
            return;
        }

        // Retrieve and print each borrowing record - O(k) where k = number of borrowing history
        for (BorrowRecord record : history) {
            System.out.println(record);
        }
    }
```

```
    // Operation 2: Record a new borrowing transaction
    public void recordNewBorrowingTransaction(String studentId, String bookId) {

        // Retrieve the student from the database - O(1)
        Student student = studentDatabase.get(studentId);

        // Check if the student exists - O(1)
        if (student == null) {
            System.out.println("Student not found. Transaction failed.");
            return;
        }

        // Retrieve the number of stocks available for the book - O(1)
        BookStock stock = bookInventory.get(bookId);

        // Check if the book is available - O(1)
        if (stock == null || stock.getQuantity() <= 0) {
            System.out.println("Book is currently not available. Transaction failed.");
            return;
        }

        // Create a new borrowing record that involves retrieving book using book ID, then decrease its quantity - O(1)
        BorrowRecord newRecord = new BorrowRecord(this, bookId);
        stock.decreaseQuantity();

        // Add the borrowing record to the student's borrowing history and print successful message - O(1)
        student.addBorrowRecord(newRecord);
        System.out.println("\nNew borrowing transaction recorded successfully.");
    }

    // Retrieve a book from the inventory by using book ID
    public Book retrieveBook(String bookId) {
        BookStock stock = bookInventory.get(bookId);
        return (stock != null) ? stock.getBook() : null;
    }
}


/**
 * The main class to demonstrate the Smart Library System.
 */
public class SmartLibBorrowingOperations {
    public static void main(String[] args) {

        // Create the system
        SmartLibSystem system = new SmartLibSystem();

        // Add books
        Book b1 = new Book("B001", "The Great Gatsby");
        Book b2 = new Book("B002", "The Catcher in the Rye");
        Book b3 = new Book("B003", "To Kill a Mockingbird");
        system.addBookToInventory(b1);
        system.addBookToInventory(b2);
        system.addBookToInventory(b3);

        // Add students
        Student s1 = new Student("S1001", "Alice Tan");
        Student s2 = new Student("S1002", "John Lee");
        system.addStudent(s1);
        system.addStudent(s2);

        // Perform borrowing operations
        system.recordNewBorrowingTransaction("S1001", "B001");
        system.recordNewBorrowingTransaction("S1001", "B002");
        system.recordNewBorrowingTransaction("S1002", "B003");

        // Retrieve their borrowing history
        system.retrieveBorrowingHistory("S1001");
        system.retrieveBorrowingHistory("S1002");
    }
}
```